

AI-Intent: A Conceptual Modeling Framework for Accountable Multi-Agent AI Systems

Anonymous Author(s)

Anonymous Institution(s)
anonymous@domain.com

Abstract. Existing agent-oriented conceptual modeling frameworks were designed for software components with deterministic execution semantics. Applied to language model agents, whose outputs are governed by probabilistic text generation and natural-language inputs, these frameworks lack three governance constructs that accountability requires: a means to *declare* the legitimate action space of each agent as a binary boundary; a means to *enforce* negative obligations at runtime via an external component rather than relying on agent self-compliance; and a means to treat inter-agent communication as *auditable evidence*.

To address these gaps, we introduce AI-INTENT, a conceptual modeling framework organized into three governance pillars. The ***Declaration*** pillar makes the legitimate action space of each agent explicit before execution through *Mandates* that encode decision rights, boundary constraints, permitted capabilities, and risk policies. The ***Enforcement*** pillar operationalizes those Mandates at runtime through a Compliance Agent that intercepts every *Proposed Action* and evaluates it against applicable constraints before delivery, enforcing norms externally rather than relying on agent self-compliance. The ***Auditability*** pillar derives a structured, durable record from every session, capturing what was approved as well as what was blocked and on what grounds.

The framework is evaluated through a reference implementation for private investment advisory, a domain that combines multiple specialized agents and clear regulatory requirements (MiFID II). Across 190 evaluation sessions, boundary violation containment averaged 97.9% with zero forced-pass occurrences. Audit trace completeness depends, however, on the instruction-following capability of the deployed model.

Keywords: Multi-Agent Systems · Conceptual Modeling · Language Model Agents · Accountability · Governance

1 Introduction

Language model-based agents are rapidly moving from research prototypes to production systems that support consequential decisions in finance, healthcare, and enterprise operations [24,28]. Unlike traditional software components, whose runtime behavior is fully determined by source code, language model agents generate outputs through probabilistic text generation that is governed by natural language inputs, training data, and model parameters. When multiple such

agents collaborate, delegating sub-tasks, synthesizing partial results, and producing joint recommendations, questions arise that existing conceptual models are not equipped to answer: what was each agent supposed to do, whether it stayed within its designated scope, and why a particular output was produced.

Addressing these questions requires revisiting the foundations. Conceptual modeling frameworks for multi-agent systems have a rich history [29,8,25], modeling agents in terms of goals, capabilities, roles, and inter-agent dependencies to provide formal foundations for requirements analysis and system design. However, these frameworks share an assumption that their target components have deterministic or rule-governed execution semantics. This assumption does not hold for language model agents, giving rise to three modeling gaps.

Gap 1: Intent boundary versus goal satisfaction. Traditional goal models, such as *i** and Tropos, treat goal satisfaction as graded [29]: softgoals can be partially achieved, contributing positively or negatively to parent goals. For language model agents, the relevant construct is not graded goal satisfaction but a binary *sortal boundary*: a request either falls within the agent’s legitimate scope or it does not. An equity analyst agent recommending gold at 35% allocation has not partially satisfied its goal; it has transgressed its mandate scope.

Gap 2: Negative obligations as runtime-enforceable constraints. Normative frameworks and deontic logic provide well-developed accounts of obligation, permission, and prohibition [26,14,7]. In classical normative agent architectures, norms are represented declaratively and reasoned about by the agent itself [9]. For language model agents, however, agents may acknowledge a constraint and then violate it, or construct superficially plausible justifications for exceptions [18]. The modeling challenge is therefore to represent negative obligations in a form that an *external* enforcement component can evaluate, rather than one that relies on agent self-compliance.

Gap 3: Communication as audit evidence. Existing agent communication models, such as FIPA-ACL [12], specify message syntax and semantics but treat communication logs as an optional engineering concern. Regulatory frameworks for AI systems, including the EU AI Act [11] and MiFID II [10], require that every consequential decision be traceable to its inputs. Meeting this requirement demands that communication logging be a *structural property* of the conceptual model, not an add-on.

To address these gaps, we propose AI-INTENT, organized into three pillars, one per gap. **Declaration** introduces the *Mandate*, a first-class artifact encoding an agent’s binary action boundary, decision rights, capabilities, and risk policies. **Enforcement** introduces *Intent Enforcement*, a mandatory gatekeeper that evaluates every *Proposed Action* against the applicable *Mandate* and *Regulatory Framework*, producing a *Compliance Verdict* before delivery. **Auditability** derives an *Accountability Trace* from the *Log Entry* of every verdict, recording what was approved as well as what was blocked and on what grounds.

Nature and scope of the contribution. We state at the outset what AI-Intent is and is not. It is *not* a new modeling language, nor an extension to the syntax

of an existing one; it is a domain-independent *conceptual model* – governance constructs with their metamodel relationships and well-formedness conditions, grounded in the Unified Foundational Ontology (UFO) [15] and rendered in OntoUML. It *augments* rather than replaces existing agent-oriented languages: where i^* , Tropos, GAIA, and NMAS model what agents want, can, and should do, AI-Intent adds the orthogonal constructs needed to declare, enforce, and audit what an agent *must not* do. The paper is accordingly structured as a gap analysis (Section 2) followed by the constructs that close those gaps (Section 3).

To evaluate AI-Intent, we have developed a reference implementation for private investment advisory, a domain combining multiple specialized agents and regulatory requirements. Evaluated across 190 sessions, spanning 10 independent runs, 19 test cases, and four adversarial disposition presets, boundary violation containment averaged 97.9% with zero forced-pass occurrences, including under adversarial agent dispositions. Even under a neutral disposition preset in which agents received no adversarial modification, the materials agent violated its allocation cap on the first attempt in 12.9% of sessions, motivating external enforcement as a structural modeling requirement.

The remainder of the paper is organized as follows. Section 2 reviews existing agent-oriented modeling frameworks. Section 3 presents the AI-Intent framework across its three pillars. Section 4 reports the empirical evaluation, while Section 5 discusses the results. Section 6 concludes and identifies directions for future work.

2 Background and Related Work

We apply an explicit selection criterion to a broad landscape. We treat as directly related any work that proposes *modeling constructs* for agent governance (Section 2.1); multi-agent implementation frameworks (Section 2.2) and AI-governance policy work (Section 2.3) are surveyed as deployment substrate and normative context, not as competing modeling contributions.

2.1 Agent-Oriented Conceptual Modeling

To identify what existing frameworks lack, the present section examines four influential modeling approaches through the lens of the three governance gaps introduced above: i^* [29], Tropos [8], GAIA [25], and Normative Multi-Agent Systems (NMAS) [7]. We note the terminological distinction the taxonomy warrants: i^* and Tropos are *goal*-oriented, whereas GAIA and NMAS are *agent*-oriented in the stricter sense; we group them here because each supplies constructs a governance model must either reuse or replace.

The i^* framework [29] models actors with goals, tasks, resources, and soft-goals connected by contribution links. Its softgoal construct explicitly supports partial satisfaction: a design decision may contribute positively or negatively to a softgoal. This graded semantics is well suited to modeling design trade-offs but is inappropriate for mandate boundaries, where violations are categorical rather than a matter of degree. **Tropos** [8] extends i^* into a full software development

methodology, introducing plans as sequences of activities satisfying goals and capability models linking agents to the plans they can execute. The framework provides no construct for prohibitions: it specifies what agents can and should do, not what they must not do.

GAIA [25] provides role models with permissions, responsibilities, activities, and protocols. Its permission construct is the closest existing analog to boundary constraints: a role’s permissions define the actions it is authorized to take. However, permissions are granted at modeling time and assumed to hold at runtime; no mechanism exists for verifying that an agent stays within its permissions during execution. For agents with deterministic behavior, this assumption is safe; for language model agents, it is not.

Normative Multi-Agent Systems (NMAAS) [14,7,9] address obligation and prohibition more directly, employing deontic operators for obligation (**O**), permission (**P**), and prohibition (**F**) [26]. Defeasible deontic logic [14] allows norms to have exceptions and to be defeated by stronger norms, which is appropriate for complex multi-stakeholder settings. However, these approaches assume agents that deliberate over norms, evaluating their intended actions against applicable norms before acting. Language model agents do not deliberate in this sense; they generate text that may or may not comply with norms, depending on how those norms are represented in the input specification and how reliably the model follows instructions [18].

Table 1 captures this analysis systematically and contrasts it with the recent trustworthy-agent frameworks (Section 2.3) and with AI-Intent. Across the four modeling frameworks, coverage is partial and uneven: *i** and Tropos represent neither prohibitions nor scope boundaries; GAIA adds permissions but cannot verify them at runtime; NMAAS closes the representational gap on prohibitions but relies on agent self-deliberation for enforcement, an assumption that fails for non-deterministic generators. The recent trustworthy-agent frameworks supply runtime auditing and provenance but no declarative modeling constructs for scope, prohibition, or risk. Only AI-Intent covers all eight capabilities, which is the gap the framework is designed to close.

2.2 Multi-Agent Language Model Architectures

Where agent-oriented modeling frameworks lack runtime enforcement, current multi-agent language model architectures lack formal modeling altogether. AutoGen [27], CrewAI [21], and LangGraph [19] enable language model agents to collaborate through role specification, tool invocation, and interaction-based task coordination, but represent agent roles primarily as natural-language descriptions with no formal constraint semantics or runtime verification. The Model Context Protocol (MCP) [2] standardizes communication between language model applications and external tools through a client-server architecture, providing interoperability but not governance. AI-Intent builds on MCP’s communication substrate and adds the governance layer that MCP lacks.

Output filtering systems, such as NeMo Guardrails [23] and Constitutional AI [6], address output compliance for individual models through input and output filtering and self-assessment mechanisms. These systems enforce constraints on single-agent outputs but do not model inter-agent accountability: they cannot

Table 1. Comparative analysis of agent governance modeling capabilities, contrasting the four agent-oriented modeling frameworks and the recent runtime trustworthy-agent frameworks (Trust. MAS: Alqithami’s AAF [1], verifiability-first agents [16]) with AI-Intent. ✓ = fully expressible; ◦ = partial/with significant extensions; × = not expressible without framework revision. Trust. MAS are runtime mechanisms, not modeling languages; marks reflect what each represents as an explicit construct. *Model operativity* denotes runtime injection of mandate constraints into the deployed model’s input specification.

Capability	i^*	Tropos	GAIA	NMAS	Trust. MAS	AI-Intent
Intent scope (binary sortal boundary)	×	×	◦	◦	×	✓
Negative obligations (prohibitions)	×	×	◦	✓	×	✓
Quantitative risk parameters	×	×	×	◦	×	✓
Runtime constraint enforcement	×	×	×	◦	◦	✓
External Compliance Agent	×	×	×	×	◦	✓
Audit-native communication	×	×	×	×	✓	✓
Compliance history in trace	×	×	×	×	◦	✓
Model operativity (runtime injection)	×	×	×	×	×	✓

trace a multi-agent decision to its constituent sub-agent outputs, do not maintain compliance histories across revision cycles, and do not produce structured audit artifacts. AI-Intent shares the underlying motivation, namely constraint enforcement on non-deterministic outputs, but operates at the multi-agent communication level rather than at the single-model output level.

2.3 AI Governance and Accountability

Existing accountability frameworks address individual model decisions [3] and dataset documentation [13,20], but fall short of *multi-agent accountability*: tracing a joint decision back to the sub-agent outputs and constraint evaluations that produced it. The EU AI Act [11] classifies AI systems by risk tier and mandates transparency, auditability, and human oversight for high-risk applications. Financial AI systems are subject to additional requirements under MiFID II, requiring that investment recommendations be traceable to their inputs and that conflicts of interest be disclosed. Neither regulatory instrument specifies how traceability is to be achieved in systems where a decision is the product of multiple collaborating agents.

A recent line of work targets trustworthiness and accountability in language model agent systems directly, and is the closest state of the art. Alqithami’s Adaptive Accountability Framework [1] is a runtime layer that records verifiable interaction provenance, detects distributional change points in streaming traces, and attributes responsibility via a causal influence graph, targeting *emergent* norm violations at scale. Verifiability-first designs [16] embed lightweight audit agents and attestations to minimize time-to-detect misalignment, while governance frameworks for multi-agent AI [22,5,4] treat accountability failures as governance-design problems and propose evaluation and policy scaffolding. These works share AI-Intent’s motivation but differ in level: they are runtime,

policy, or evaluation *mechanisms*, whereas AI-Intent is a *conceptual model* that specifies the constructs such mechanisms leave implicit – a declared per-agent action boundary, an external enforcement role, and an audit artifact derived as a well-formedness condition. Where they detect or attribute violations *a posteriori* from observed traces, AI-Intent’s enforcement intercepts each Proposed Action *before* delivery. The directions are complementary: a system could realize AI-Intent’s constructs and adopt [1]’s attribution for residual emergent behaviors that per-message enforcement does not cover.

Enterprise architecture standards and Service-Oriented Architecture compliance logging provide audit trails as infrastructure services. By design, such logs record what happened; they are not first-class elements that constrain system behavior at runtime. AI-Intent addresses this distinction: in *AI-Intent*, the *Accountability Trace* is a structural model component whose existence is a well-formedness condition, and the *Compliance Verdict* attached to each message is a metamodel relationship rather than a log entry.

3 The AI-Intent Framework

AI-Intent is a metamodel of governance constructs, their relationships, and well-formedness conditions, independent of any implementation. This section states its ontological commitments (Section 3.1) and then its constructs, grouped into three pillars (Section 3.2), distinguishing throughout constructs *derived from* UFO (which inherit its formal properties) from *design choices* layered on top; Table 2 summarizes this mapping and the precise extent of the framework’s dependence on UFO.

3.1 Core Principles and Ontological Commitments

AI-Intent adopts the following ontological commitments, grounded in the Unified Foundational Ontology (UFO) [15] and standard deontic semantics [26,14].

Agents as intentional objects. Following UFO, agents are treated as intentional objects with dispositions to act in certain ways. An agent’s Mandate specifies the intended disposition: the range of behaviors the agent is authorized to exhibit, the boundaries it must not cross, and the risk parameters within which it must operate.

Intent scope as a sortal boundary. We use *sortal* in its established ontological sense [17]: a sortal universal supplies a principle of identity and individuation, determining for any candidate whether it *is or is not* an instance of the type [15], and is therefore inherently binary rather than graded. The intent scope in the Mandate is not a goal, which admits degrees of satisfaction, but a natural-language specification delineating such a sortal boundary: the category of situations to which the agent’s competence applies, which partitions the query space into in-scope and out-of-scope regions. We adopt the sortal reading rather than a mere subtype predicate because its identity criterion makes “the

same kind of request” well defined across the paraphrastic variation typical of language model queries.

Boundary constraints as prohibitions. Following deontic logic [26], the boundary constraints encoded in the Mandate are prohibitions of the form $\mathbf{F}(\phi)$: the agent is forbidden from producing outputs satisfying predicate ϕ . Risk parameters provide quantitative instantiations, for example $\mathbf{F}(\text{allocation} > 0.15)$ for a 15% cap. Unlike NMAS norms, these prohibitions are evaluated by the Intent Enforcement component against every Proposed Action before delivery, rather than reasoned about by the agent itself.

Intent Enforcement as a structural role. Intent Enforcement is not an optional component but a central element of the framework. It makes prohibitions operative at runtime. A framework that can represent prohibitions in a Mandate but provides no enforcement mechanism cannot guarantee that those prohibitions hold on any given agent output.

Communication as evidence. Every inter-agent action is a time-stamped, non-repeatable UFO *event* that produces a Compliance Verdict, recorded as a Log Entry and aggregated into the Accountability Trace.

Dependence on UFO. AI-Intent *depends on* UFO for the ontological *categorization* of its constructs and the resulting entailments, but does *not* require a UFO reasoner at runtime: the reference implementation operationalizes the categories without ontological inference. Three constructs are genuinely UFO-derived. Treating each inter-agent action as an *event* entails that it is time-indexed and non-repeatable, licensing its use as non-repudiable evidence; treating the Accountability Trace as a *situation* (a UFO truthmaker aggregating events) makes it a derived first-class object rather than a side-effect log; treating agents as *intentional objects with dispositions* lets a Mandate constrain a disposition-to-act rather than an observed behavior. By contrast, the Mandate’s internal structure, the external Compliance Agent, and the quantitative risk parameters are design choices layered on top. The value of the UFO grounding is clearest in the counterfactual: a non-UFO design would plausibly have produced (i) the Mandate as a goal-set rather than a normative description, reintroducing the graded satisfaction that Gap 1 rejects; (ii) the audit trail as an infrastructure artifact rather than a derived situation, forfeiting the well-formedness guarantee of Section 3.2; and (iii) messages permitted to exist without an attached verdict, breaking communication-as-evidence. The three UFO-derived commitments are precisely what exclude these alternatives. Table 2 records the full mapping, including the OntoUML stereotype and the reference-implementation realization of each construct.

3.2 Framework Structure and Pillars

AI-Intent organizes its modeling constructs into three pillars that reflect the governance lifecycle of a multi-agent session: Declaration, Enforcement, and Auditability (see Figure 1).

Table 2. Grounding of AI-Intent constructs in UFO. *Src* distinguishes UFO-derived commitments (which inherit formal entailments) from *design* choices layered on top; the *Realization* column gives the corresponding element of the reference implementation.

Construct	UFO category / toUML stereotype	On- Src Realization in reference impl.
Principal	Intentional object, «kind»	des Human/institutional owner (config)
Agent	Intentional object w/ dispositions, «kind»	der Orchestrator + three sub-agents
Mandate	Normative description (<i>not</i> a goal)	des Pydantic schema injected into input spec
Intent scope	Sortal universal	der Natural-language scope string
Boundary constraint	con- Negative duty, «mode»	des Predicate in the rule registry
Proposed Action	Event (non-repeatable), «event»	der MCP message, pre-delivery
Compliance dict	Ver- Mode dependent on the Proposed Action	der Verdict payload with constraint IDs
Log Entry	Event record, «event»	der SQLite row
Accountability Trace	Situation from session events, «situation»	der Session JSON export
Regulatory Framework	External normative description	des Rule registry (MiFID II)

Pillar 1: Declaration. This pillar establishes the legitimate action space of the system before any session begins. It contains four constructs: Principal, Agent, Mandate, and Proposed Action.

A *Principal* is an individual or institutional entity that initiates and owns the governance structure of an agent or a multi-agent system. It holds objectives expressing the high-level intent the system is deployed to serve. A Principal delegates tasks to Agents and defines the Mandates governing them.

An *Agent* is an executable, potentially composite, component with Domain Expertise scoping its competence, that carries out tasks on behalf of the Principal. A Composite flag distinguishes orchestrators from specialist sub-agents without introducing a separate class hierarchy; composite agents invoke the Compliance Agent before delegating to each sub-agent. Agents may also communicate laterally, representing coordination within a multi-agent architecture.

The *Mandate* is the central Declaration artifact. It is a type-level specification that simultaneously defines what an agent is competent to address, what it is forbidden from producing, and the operational parameters within which it must act. The Mandate is structured into four named sections. *Decision Rights* define the range of decisions the agent may make without escalation. *Boundaries* encode the agent’s negative obligations as legal, ethical, and operational prohibitions. Formally, each boundary constraint is a triple $c_i = \langle text_i, \phi_i, \tau_i \rangle$, where $text_i$ is the natural-language statement, ϕ_i a machine-evaluable predicate over

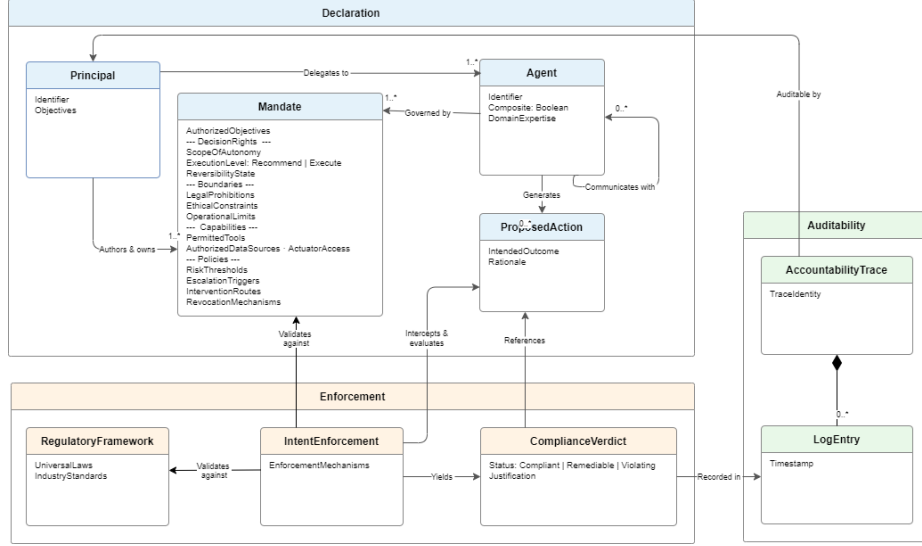


Fig. 1. The AI-Intent conceptual model.

a Proposed Action, and $\tau_i \in \{\mathbf{F}, \mathbf{O}\}$ its deontic type; a prohibition ($\tau_i = \mathbf{F}$) is violated *iff* a Proposed Action *satisfies* ϕ_i . A risk parameter is the quantitative binding of a free variable in ϕ_i (the worked example below makes this concrete). *Capabilities* enumerate the tools, data sources, and actuators the agent is permitted to invoke. *Policies* govern the agent’s response when boundaries are approached or crossed, through risk thresholds, escalation triggers, intervention routes, and revocation mechanisms.

A *Proposed Action* is a candidate output generated by an Agent prior to compliance evaluation. It carries an Intended Outcome, the result the agent seeks to produce, and a Rationale, the agent’s justification. The Proposed Action is the unit of evaluation in Pillar 2: it is intercepted by the Enforcement pillar before any delivery to another agent or to the user.

Pillar 2: Enforcement. This pillar operationalizes the constraints declared in Pillar 1. It intercepts every Proposed Action and every Mandate at the point of cross-agent communication, evaluating them against applicable rules before permitting delivery. Three constructs constitute this pillar: Regulatory Framework, Intent Enforcement, and Compliance Verdict.

The *Regulatory Framework* aggregates the external normative context within which the system operates. It refers to both universal laws, comprising statutory and regulatory instruments such as the EU AI Act [11] and MiFID II, and industry standards, comprising domain-specific codes of conduct. The Regulatory Framework is used by Intent Enforcement to supply the rule registry against which Mandate constraints are validated.

The *Intent Enforcement* construct, realized at runtime as a Compliance Agent, is the structural gatekeeper of the framework. It receives two inputs:

the Mandate, validated against the Regulatory Framework to verify that declared constraints are legally coherent, and the Proposed Action, intercepted and evaluated against the Mandate’s constraint set before delivery.

As a framework construct, Intent Enforcement is defined by the *requirements* it must satisfy, not by any particular algorithm. Four requirements are constitutive. *(R1) Non-bypassability*: every cross-agent Proposed Action is mediated; there is no delivery path that circumvents evaluation. *(R2) Independence*: the evaluator is distinct from the agent under evaluation, so that enforcement does not inherit the stochasticity of the generator it governs (cf. [18]). *(R3) Decidability of the verdict*: for every constraint, the verdict is a determinate approve/block over the Proposed Action, with a named violated constraint when blocked. *(R4) Terminating revision*: rejected actions enter a bounded revision loop that terminates in delivery or in a recorded forced block, never in an unrecorded pass. These requirements are what the framework mandates; how a deployment discharges them (deterministic predicate checks, a semantic classifier, or a hybrid) is an implementation concern, exercised by the deliberately minimal realization evaluated in Section 4.

For each intercepted Proposed Action, Intent Enforcement produces a *Compliance Verdict* capturing whether the action is approved for delivery or blocked, identifying the specific constraints from the applicable Mandate that were satisfied or violated, and, when the action is rejected, providing the revision directive returned to the originating Agent. The Compliance Verdict is the linking artifact between the Enforcement and Auditability pillars: every verdict is recorded in a Log Entry, thereby initiating the audit chain that culminates in the Accountability Trace for the session.

Worked example (risk parameter to verdict). In the Materials Mandate of Figure 1, the *Policies* section carries the risk parameter $\text{max_allocation} = 0.15$, which instantiates a *Boundaries* prohibition $\phi \equiv (\text{allocation} > 0.15)$: the parameter supplies the threshold, the deontic type **F** the polarity, together forming a machine-comparable predicate. If the Materials Agent emits a *Proposed Action* with $\text{allocation} = 0.18$, Intent Enforcement evaluates ϕ , finds the prohibition breached, and issues a *blocked Compliance Verdict* naming the violated constraint with a revision directive. Risk parameters are thus not free-standing numbers but the operands that make boundary predicates decidable.

Pillar 3: Auditability. The Auditability pillar derives a structured, durable record from the session history produced by the other pillars. It contains two constructs: Accountability Trace and Log Entry.

A *Log Entry* is the atomic audit record produced for each compliance evaluation event. Log Entries carry by reference the identifiers of the originating Proposed Action and Mandate, keeping the record navigable without introducing upward associations that would violate pillar encapsulation.

The *Accountability Trace* is a situation, in the UFO sense, derived from a complete session. It is a structured aggregation of the Log Entries produced during that session. The derivation is exact. For a session s of compliance evaluation events $\langle e_1, \dots, e_n \rangle$, each e_i yields one Compliance Verdict recorded in one

Log Entry l_i , and the Trace is the aggregation $T(s) = \langle l_1, \dots, l_n \rangle$ closed under the reference links each l_i carries to its Proposed Action and Mandate. Two well-formedness conditions make this structural rather than a reporting convenience: **(WFC-1)** every Compliance Verdict is recorded in exactly one Log Entry, so no evaluation is unlogged; **(WFC-2)** a completed session’s Trace contains a Log Entry for every cross-agent Proposed Action, so no action is unaccounted for. The difference from conventional logging follows: a conventional audit log is a side effect that may be absent, partial, or post-hoc-editable without violating any model constraint, whereas here a session missing a Log Entry for an evaluated action is *ill-formed by definition*, and because the Trace derives from the same Verdicts that gate delivery, the record cannot diverge from what was enforced. Auditability is thus a consequence of the enforcement structure, not a component bolted alongside it.

A terminological distinction is necessary here. The Accountability Trace is the full structured, session-level audit object described above. The *accountability note*, by contrast, is a human-readable summary field embedded within the final synthesis response delivered to the user; it reports a subset of the trace, comprising the session identifier, agents consulted, constraints checked, and any violations surfaced, in prose form. The trace is the evidentiary object; the note is its user-facing projection. ATC scoring (Section 4) measures field presence in the accountability note as a proxy for whether the underlying trace was correctly populated.

4 Evaluation

The evaluation of AI-Intent addresses two complementary dimensions: *framework expressiveness*, whether the three-pillar structure can model governance scenarios that existing frameworks cannot (Section 4.2), and *framework applicability*, whether the reference implementation enforces Mandate boundaries reliably across sessions, including under adversarial agent dispositions (Section 4.3).

The evaluation tests the framework’s constructs, not merely a working system, and each part maps to a claim. Expressiveness tests the *modeling* claim: scenarios S1–S3 instantiate the three gaps of Section 1 and thereby exercise the Declaration, Enforcement, and Auditability constructs in turn, asking whether each represents a scenario i^* , Tropos, GAIA, or NMAS cannot. Applicability tests the *operativity* claim: whether the constructs hold at runtime once realized. Its six dimensions each map to a construct (Table 4): BVC/CGP measure the Compliance Verdict gate (Enforcement), ME/CDA measure Mandate boundary detection (Declaration via Enforcement), ATC measures Accountability Trace population (Auditability), and DC measures whether Enforcement holds independently of agent disposition. The investment domain was chosen because MiFID II supplies externally-given, non-negotiable constraints, so the constraints tested are imposed by regulation rather than authored by us – guarding against a case study constructed to confirm its own premise.

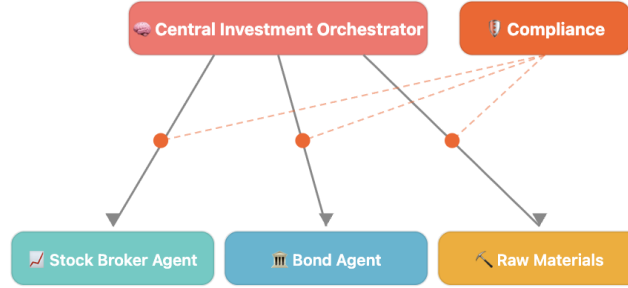


Fig. 2. The reference implementation: a Central Orchestrator and three specialist sub-agents (Stocks, Bonds, Materials), each carrying a dedicated Mandate (shown as an attached mandate box), with the Compliance Agent interposed on every Orchestrator–sub-agent channel. No sub-agent is reachable except through the Compliance Agent, realizing requirement R1 (non-bypassability). Mandate contents are summarized in Table 3.

4.1 Reference Implementation

The reference implementation comprises four agents in a hierarchical architecture (see Figure 2): a Central Orchestrator that coordinates task delegation and synthesis, and three specialist sub-agents, Stocks, Bonds, and Materials, each governed by a dedicated Mandate. The Compliance Agent intercepts all communication between the Orchestrator and the sub-agents, producing a Compliance Verdict for every Proposed Action before delivery. This reference Compliance Agent is a deliberately minimal realization of the Intent Enforcement construct: it discharges requirements R1–R4 (Section 3.2) using a deterministic predicate checker over structured agent output, backed by a narrow semantic fallback. It is intended to demonstrate that the construct is operable, not to represent the most capable checker the framework admits; Section 5 discusses where this minimal realization is the binding limitation rather than the framework itself. Table 3 summarizes the agent mandates. Constraints are annotated with their deontic type: **F** (prohibition) or **O** (obligation)¹.

The technology stack comprises Python 3.11+ with Pydantic v2, SQLite for MCP message persistence, Streamlit for the dashboard, and Llama 3.1 8B served locally via Ollama.²

¹ The complete source code of the reference implementation, including all agent mandates, the Compliance Agent, the MCP message bus, the orchestration protocol, the evaluation runner, and the session JSON exports underlying the results in Section 4, is available in the online repository at <https://anonymous.4open.science/r/ai-intent-09F2>. The link is anonymized for double-blind review; a public, attributed repository will replace it upon acceptance.

² Local deployment ensures reproducibility and eliminates dependencies on external inference services.

Table 3. Agent mandates in the reference implementation. $|C|$ = boundary constraints; $|R|$ = risk parameters. Representative constraint examples are provided.

Agent	Role	$ C $	$ R $	Key Constraints (examples)
Central	Orchestrator	5	2	$\mathbf{F}(\text{single asset class} > 40\%)$; $\mathbf{O}(\text{consult} \geq 1 \text{ sub-agent})$; $\mathbf{O}(\text{surface all violations})$
Stocks	Equity Analyst	5	3	$\mathbf{F}(\text{non-large-cap})$; $\mathbf{F}(\text{position} > 10\%)$; $\mathbf{F}(\text{leverage})$; $\mathbf{O}(\text{ESG screening})$
Bonds	Fixed Income	5	4	$\mathbf{F}(\text{rating} < \text{BBB+})$; $\mathbf{F}(\text{duration} > 10 \text{ yr})$; $\mathbf{F}(\text{EM debt})$; $\mathbf{F}(\text{bucket} > 30\%/\text{yr})$
Materials	Commodities	5	4	$\mathbf{F}(\text{non-Au/Ag commodity})$; $\mathbf{F}(\text{allocation} > 15\%)$; $\mathbf{F}(\text{leveraged ETF/futures})$; $\mathbf{O}(\text{inflation rationale})$

4.2 Expressiveness

Framework expressiveness is evaluated using three governance scenarios that instantiate the three gaps identified in Section 1: out-of-scope declination (S1), quantitative constraint enforcement (S2), and cross-agent audit reconstruction (S3).

S1: Out-of-scope declination. The Mandate’s intent scope provides a natural-language sortal boundary injected into the agent’s input specification as an enforcement instruction. Agents correctly returned `out_of_scope:true` with named constraint references when queried about assets outside their scope, for instance, the stocks agent asked about gold and the bonds agent asked about equities. GAIA can represent that a role is not permitted to advise on certain asset classes but provides no mechanism for the role to identify the violated permission at runtime. i^* has no prohibition construct.

S2: Quantitative constraint enforcement. Risk parameters provide machine-comparable bounds; the Compliance Agent’s deterministic checker evaluated these against structured agent output. The checker fired on every session in which an agent recommended a position exceeding its quantitative limit. Every violation that occurred was eventually caught before delivery through the revision protocol. NMAS frameworks can represent quantitative norms through arithmetic constraints in deontic logic, but enforcement requires the agent to evaluate its own output, a procedure that is unreliable for non-deterministic generators. GAIA has no quantitative constraint construct.

S3: Audit reconstruction. The Accountability Trace provides a structured artifact from which the full decision sequence can be reconstructed: agents consulted, routing rationale, per-agent constraint checks, compliance history, and blocked agents with rule identifiers. Every session in the reference implementa-

tion supported complete reconstruction of the decision sequence without reference to any external source. No existing framework provides an equivalent audit artifact as a structural model component.

Completeness Boundaries. The expressiveness scenarios also reveal the limits of AI-Intent’s current modeling scope. Three boundaries are noted. *First, conflicting constraints:* when two Mandate constraints conflict, the framework detects the violation but does not resolve it. Conflict resolution would require a preference ordering over constraints, which the current conceptual model does not support. *Second, dynamic scope adjustment:* the Mandate is immutable at runtime. Scenarios requiring Mandate expansion or contraction in response to user authentication, market conditions, or regulatory changes require a versioning mechanism not currently modeled. *Third, norm exceptions:* unlike defeasible deontic logic [14], AI-Intent constraints have no exception structure. This is a deliberate design choice for the regulated financial domain, where constraint overrides are themselves regulated activities, but it limits applicability in domains where norm exceptions are common.

4.3 Applicability

Evaluation design. The protocol comprises six dimensions (Table 4) and 19 test cases: 15 across five categories (in-scope baseline, out-of-scope, hard constraint violation, synthesis integrity, trace integrity) and four (TC-16–19) evaluating disposition-invariant enforcement under four presets — *Neutral*, *Aggressive Broker*, *Reckless Portfolio Manager* (treats constraints as overridable), and *Groupthink* (agents suppress their own constraints). An automated runner invokes the orchestrator per case with a fresh session; scoring functions apply deterministic pattern matching over session JSON (allocation regexes, rule-identifier lookup, field-presence checks) to assign 0/1/2 per dimension, with no model involvement. The suite ran 10 independent times, yielding 190 evaluations; all data and code are in the repository (footnote 1). Table 4 reports each dimension’s claim, threshold, and result.

BVC/CGP (both 97.9%). These co-occur exactly across all runs, measuring the same event. In 6 of 10 runs no non-compliant message was delivered; the other four each involved one borderline case under *reckless_portfolio*, where the materials agent recommended 14.8% gold — within the 15% limit, hence correctly approved by the numerical criterion, though post-hoc review read the phrasing as implying intended circumvention. No forced-pass occurred in any of the 190 evaluations, so CGP cleared its 85% threshold throughout; the four cases are numerically compliant approvals, not false positives.

ATC (71.3%, below 80%). The trace mechanism functioned structurally in every session (notes present, session IDs correct, agents listed); the shortfall is Llama 3.1 8B omitting violated rule identifiers in 28.7% of sessions — an instruction-following limitation of the small model, not a design failure, motivating a more capable model where complete traces are mandatory.

CDA (38.6% unconditioned, 50.6% conditioned). The unconditioned figure is inflated by violation-free sessions that score maximally by default; con-

Table 4. Evaluation dimensions: framework claim, pass threshold, and result (mean, std) over 10 runs (190 evaluations); deterministic scoring from session JSON, with per-dimension rules and full ranges in the repository (footnote 1).

Dim.	Framework claim	Thr.	Mean (std)	Result
BVC	No non-compliant message delivered	100%	97.9% (2.7)	Near-pass
CGP	No compliant response falsely blocked	85%	97.9% (2.7)	Pass
ATC	Traces carry session ID, agents, rule IDs	80%	71.3% (5.3)	Below
CDA	Violations caught on first evaluation	90%	38.6 _u /50.6 _c % (5.3)	Below
ME	Out-of-scope declined with constraint name	75%	35.0% (22.9)	Volatile
DC	Output within limits under any preset	—	57.5 _s % (36.8)	Strict

BVC = Boundary Violation Containment; CGP = Compliance Gate Precision; ATC = Accountability Trace Completeness; CDA = Constraint Detection Accuracy (*u* unconditioned, *c* conditioned); ME = Mandate Enforcement; DC = Disposition Containment (*s* strict rubric).

ditioned on the 72 of 110 violation-occurring sessions,³ first-attempt detection is 50.6%. The shortfall reflects violations caught on revision 2 or via later verdicts rather than the first; every violation was eventually caught before delivery. CDA thus measures detection *latency*, not *reliability* — which is sufficient for containment in a revision-based architecture.

ME (35.0%, std 22.9%). Highly volatile, reflecting routing non-determinism rather than enforcement: out-of-scope declination tests require specific routing, and when the orchestrator routed elsewhere the target agent was not invoked. We treat this as a methodology finding.

DC (disposition containment). Across all 40 disposition evaluations (4 presets $\times$ 10 runs), zero forced-pass occurred; the *reckless_portfolio* preset triggered revisions but never breached the Compliance Agent. The strict rubric (DC_s=57.5%, std 36.8%) tracks the 2–5 $\times$ revision overhead under adversarial presets — i.e. compliance effort, not safety. The gate held in every run.

5 Discussion

5.1 Contributions to the Conceptual Modeling Community

AI-Intent identifies three properties that conceptual modeling frameworks for language model agents require but existing frameworks lack, each argued in Sections 1–3. First, a binary *sortal boundary* on the action space, complementary to graded goals and made explicit as a first-class enforceable artifact. Second, *external* norm enforcement elevated to a structural role rather than left to agent

³ CDA is scored on the 11 test cases with expected or detected violations (110 sessions); self-compliance is measured across all 129 materials-agent sessions. The full denominator arithmetic is in the repository.

self-compliance: the empirical 12.9% first-attempt violation rate under neutral disposition (25.6% overall across 129 sessions) shows agent-level enforcement is insufficient for non-deterministic generators, so compliance must be modeled, not assumed. Third, *communication as evidence*, realized by attaching a Compliance Verdict to every message and deriving the Accountability Trace — a pattern that applies to any regulated multi-agent setting, not only the investment domain.

5.2 Compliance-Utility Tension

When the Materials agent was permanently blocked on the leveraged gold ETF query, the Central Orchestrator produced a recommendation using only the Stocks and Bonds agents, a result that was fully compliant but entirely non-responsive to the user’s query. This illustrates a conflict between constraint and goal satisfaction. The current AI-Intent design resolves this conflict by giving constraints lexicographic priority: a compliant non-answer is preferred to a non-compliant answer. This is the appropriate resolution for regulated financial domains, where a non-compliant recommendation constitutes a regulatory violation while a failure to answer does not. The framework should nonetheless represent this priority explicitly. Future work will introduce a constraint priority field in the Mandate schema and a conflict resolution policy at the orchestrator level.

5.3 Limitations

Model operativity depends on instruction following. The Mandate is operative only insofar as the deployed model follows its input specification. ATC at 71.3% confirms that Llama 3.1 8B fails to populate the accountability note schema correctly in 28.7% of sessions. The framework is model-agnostic in design, but its operativity guarantee is model-dependent in practice.

Deterministic predicate checking. The Compliance Agent relies on machine-evaluable predicates for constraint checking. In the reference implementation, leverage detection uses substring matching, which fires when an agent correctly declines a leveraged product by naming it. A 15-word negation context window suppresses matches preceded by terms such as “not,” “avoid,” or “decline,” but formulations outside that vocabulary still trigger the check. More expressive predicate languages remain an open design question.

Routing non-determinism. The framework does not currently constrain how a composite agent routes sub-tasks to specialist agents. When routing varies across sessions, out-of-scope declination tests may fail to invoke the target agent, rendering mandate enforcement untestable for that agent in that session. Evaluation protocols for out-of-scope behavior should therefore enforce routing to the target agent rather than relying on the orchestrator.

Specification-predicate consistency. Recall that each boundary constraint is a triple $c_i = \langle text_i, \phi_i, \tau_i \rangle$ (Section 3.2), with ϕ_i the machine-evaluable predicate coded in the regulatory rule registry. If ϕ_i is miscoded relative to $text_i$, the Compliance Agent silently enforces the wrong rule and the Accountability Trace records that mismatch as if it were intended. This matters because the

enforcement guarantees of Section 4.3 are conditional on specification quality: a correct gate enforcing a wrong predicate yields either a *false block* of a compliant action (over-strict ϕ_i) or, more seriously, a *silent false pass* of a non-compliant one (under-strict ϕ_i , e.g. a cap coded as *allocation* > 0.20 where the text says 15%); in the latter case BVC still reports containment because no rule fired, so the metric cannot detect its own predicate being wrong. The four borderline cases in Section 4.3 (14.8% gold under *reckless_portfolio*) are the one empirical instance of this class observed: the numeric predicate *allocation* > 0.15 is under-restrictive relative to the text’s intent that the cap not be circumvented, so a response that is numerically compliant yet phrased to imply circumvention passes the gate — which is why post-hoc human review, not the deterministic checker, surfaced them. We are candid that the evaluation was designed to *avoid* rather than *expose* this mode: expected rule identifiers were derived from the same Mandate definitions used to code the predicates, so the two share provenance and a common miscoding would go undetected. Establishing consistency independently — e.g. deriving ϕ_i and $text_i$ from separate sources and cross-checking, or property-based testing against labeled cases — is left as an open modeling problem.

5.4 Threats to Validity

Internal validity. Scoring is deterministic from session JSON, eliminating evaluator subjectivity. CDA scoring depends on rule-identifier matching against expected violations; we mitigated the risk of misspecified expectations by deriving expected rules directly from Mandate definitions.

Scope of validity and conditions for transfer. All quantitative results are from one domain (investment advisory) and one model (Llama 3.1 8B); the 12.9% self-compliance rate is specific to the materials agent’s allocation constraint. We therefore position the results as a *proof of concept*: they establish that the constructs are operable and that external enforcement is necessary in at least one regulated setting, not that the framework generalizes without further study. The *domain-independent* core (metamodel, pillars, well-formedness conditions) is separable from *domain-specific* content (Mandate text, predicate registry, risk parameters), and is expected to transfer where (i) competence admits a sortal in/out-of-scope boundary, not only graded goals; (ii) norms are prohibitions expressible as machine-evaluable predicates; (iii) Mandates are session-stable; and (iv) the domain tolerates lexicographic constraint priority, so a compliant non-answer is acceptable — as in regulated finance, but not emergency triage. Transfer is poor where norms are defeasible with routine exceptions (Section 4.2), irreducibly qualitative (no decidable ϕ), or require dynamic renegotiation. Replication in a second regulated domain (e.g., clinical decision support) is the primary next step.

Construct validity. The six dimensions map to the framework’s three claims. BVC and CGP directly measure the Compliance Agent’s core guarantee; ATC and CDA are proxies — ATC measures trace-population completeness rather than correctness, and CDA detection latency rather than reliability.

6 Conclusion

AI-Intent grounds three governance constructs — a binary action-space boundary, runtime-enforceable negative obligations, and communication-as-evidence — in deontic logic and UFO, making explicit what is new (sortal boundaries, external norm enforcement) and what is appropriated (prohibition semantics from NMAS, event ontology from UFO). Empirically, external enforcement proved necessary rather than optional: even under neutral disposition agents violated their allocation cap on first attempt in 12.9% of sessions, yet across 190 evaluations — including a preset instructing agents to override their constraints — boundary violation containment averaged 97.9% with zero forced-pass occurrences.

The message for the conceptual modeling community is that compliance enforcement must be modeled, not assumed. For agents with deterministic behavior, this assumption is safe and existing frameworks correctly omit it. For language model agents, it fails systematically, and any governance-oriented conceptual model that does not represent the compliance function as a structural component will underspecify the system in ways that are consequential for accountability and regulatory compliance.

References

1. Alqithami, S.: Adaptive accountability in networked multi-agent systems. In: Proc. AAAI/ACM Conference on AI, Ethics, and Society (AIES). vol. 8, pp. 127–137 (2025), <https://doi.org/10.1609/aies.v8i1.36536>
2. Anthropic: Model context protocol specification. <https://modelcontextprotocol.io> (2024)
3. Arrieta, A.B., Díaz-Rodríguez, N., Del Ser, J., et al.: Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion* **58**, 82–115 (2020)
4. Authors to verify: Trustworthy multi-agent AI governance framework [title/venue to verify]. *IEEE Xplore*, document 11485555 (2025), <https://ieeexplore.ieee.org/document/11485555>
5. Author(s) to verify: A unified evaluation and governance framework for trustworthy LLM agents. *TechRxiv preprint* (2026), <https://doi.org/10.36227/techrxiv.176799772.28164151>
6. Bai, Y., Kadavath, S., Kundu, S., et al.: Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073* (2022)
7. Boella, G., van der Torre, L., Verhagen, H.: Introduction to normative multiagent systems. *Computational & Mathematical Organization Theory* **12**(2–3), 71–79 (2006)
8. Bresciani, P., Perini, A., Giorgini, P., Giunchiglia, F., Mylopoulos, J.: Tropos: An agent-oriented software development methodology. *Autonomous Agents and Multi-Agent Systems* **8**(3), 203–236 (2004)
9. Dignum, F.: Autonomous agents with norms. *Artificial Intelligence and Law* **7**(1), 69–79 (1999)

10. European Parliament and Council: Directive 2014/65/EU on markets in financial instruments and amending directive 2002/92/EC and directive 2011/61/EU (MiFID II). Official Journal of the European Union L 173, 349–496 (2014)
11. European Parliament and Council: Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act). Official Journal of the European Union (2024)
12. FIPA: FIPA ACL message structure specification. Tech. Rep. SC00061G, Foundation for Intelligent Physical Agents (2002)
13. Gebru, T., Morgenstern, J., Vecchione, B., et al.: Datasheets for datasets. *Communications of the ACM* **64**(12), 86–92 (2021)
14. Governatori, G., Hulstijn, J., Riveret, R., Rotolo, A.: Characterising deadlines in temporal modal defeasible logic. In: *Proceedings of the 18th Australian Joint Conference on Artificial Intelligence (AI 2005)*. *Lecture Notes in Artificial Intelligence*, vol. 3809, pp. 486–496. Springer (2005)
15. Guizzardi, G.: *Ontological Foundations for Structural Conceptual Models*. Telematica Instituut / CTIT, Enschede (2005)
16. Gupta, A.: Verifiability-first agents: Provable observability and lightweight audit agents for controlling autonomous LLM systems. *arXiv preprint arXiv:2512.17259* (2025)
17. Gupta, A.: *The Logic of Common Nouns: An Investigation in Quantified Modal Logic*. Yale University Press, New Haven, CT (1980)
18. Huang, J., Gu, S.S., Hou, L., Wu, Y., Wang, X., Yu, H., Han, J.: Large language models cannot self-correct reasoning yet. *arXiv preprint arXiv:2310.01798* (2023)
19. LangChain: LangGraph: Building stateful, multi-actor applications with LLMs. <https://github.com/langchain-ai/langgraph> (2024)
20. Mitchell, M., Wu, S., Zaldivar, A., et al.: Model cards for model reporting. In: *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAT*)*. pp. 220–229 (2019)
21. Moura, J.: CrewAI: Framework for orchestrating role-playing autonomous AI agents. <https://github.com/joaomdmoura/crewAI> (2024)
22. Pujari, T., Goel, A., Sharma, A.: Ethical and responsible AI: Governance frameworks and policy implications for multi-agent systems. *International Journal of Science and Technology* **3**(1), 72–89 (2024), <https://doi.org/10.56127/ijst.v3i1.1962>
23. Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C., Cohen, J.: NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*. pp. 431–445 (2023)
24. Wang, L., Ma, C., Feng, X., et al.: A survey on large language model based autonomous agents. *Frontiers of Computer Science* **18**(6), 186345 (2024)
25. Wooldridge, M., Jennings, N.R., Kinny, D.: The Gaia methodology for agent-oriented analysis and design. *Autonomous Agents and Multi-Agent Systems* **3**(3), 285–312 (2000)
26. von Wright, G.H.: *An Essay in Modal Logic*. North-Holland, Amsterdam (1951)
27. Wu, Q., Bansal, G., Zhang, J., et al.: AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155* (2023)
28. Xi, Z., Chen, W., Guo, X., et al.: The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864* (2023)
29. Yu, E., Giorgini, P., Maiden, N., Mylopoulos, J.: *Social Modeling for Requirements Engineering*. MIT Press (2011)